



VISUAL PROGRAMMING

Layout Panes and Groups

MUSBAH J. MOUSA
IYAS A. I. ESHAIKHKHALIL

OUTLINES

- **Groups**
- **Layout Panes**
 - StackPane
 - FlowPane
 - GridPane
 - BorderPane
 - HBox
 - VBox
 - Anchor

GROUP CLASS

- The Group class is often used to group nodes and to perform transformation and scale as a group.

What is your position?

☒ Teacher
☐ Student

Select your interests:

☒ Software Engineering	☐ Desktop Development
☐ Mobile Development	☒ DevOps
☐ Web Development	☒ Security and Networking

Groups

LAYOUT PANES

- A JavaFX application **can manually** lay out the UI by setting the position and size properties for each UI element.
- An **easier** option is to make use of layout panes.
- The JavaFX SDK provides several layout panes for the easy setup and management of classic layouts such as rows, columns, stacks, tiles, and others.
- As a window is resized, the layout pane automatically repositions and resizes the nodes that it contains **according to their preferred size range preferences**.

LAYOUT PANES

- **Important Notes:**
 - **Panes and UI control objects are resizable.**
 - **Group, Shape, and Text objects are not resizable.**

LAYOUT PANES

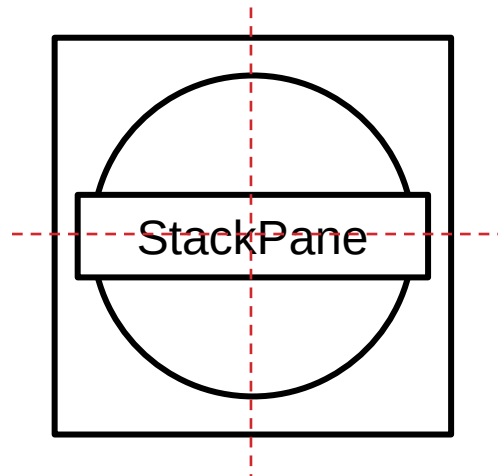
- **UI controls have three values for it's size:**
 - Preferred size
 - Min size
 - Max size
- **By default, UI controls compute default values for their preferred size that is based on the content of the control.**
 - For example, the computed size of a Button object is determined by the length of the text and the size of the font used for the label, plus the size of any image.
- **The computed size is just big enough for the control and the label to be fully visible.**

BUILT-IN LAYOUT PANES

Class	Description
Pane	Base class for layout panes. It contains the <code>getChildren()</code> method for returning a list of nodes in the pane.
StackPane	Places the nodes on top of each other in the center of the pane.
FlowPane	Places the nodes row-by-row horizontally or column-by-column vertically.
GridPane	Places the nodes in the cells in a two-dimensional grid.
BorderPane	Places the nodes in the top, right, bottom, left, and center regions.
HBox	Places the nodes in a single row.
VBox	Places the nodes in a single column.
Anchor	Enables you to anchor nodes to the top, bottom, left side, right side, or center of the pane.

STACK PANE

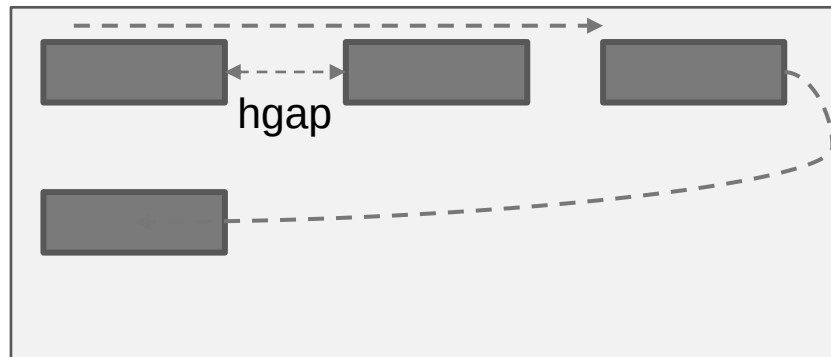
- Places the nodes on top of each other in the center of the pane.



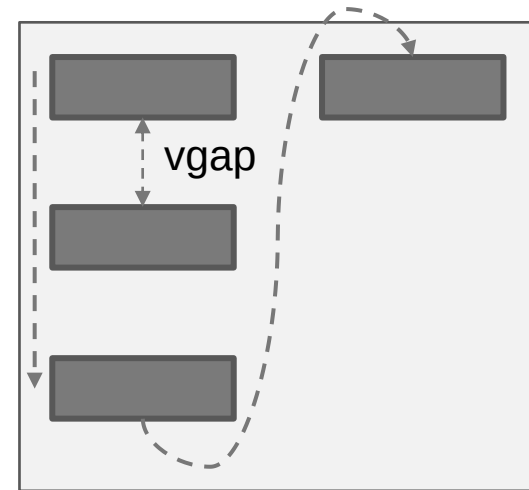
FLOW PANE

- **FlowPane** arranges the nodes in the pane horizontally from left to right, or vertically from top to bottom, in the order in which they were added.
- When one row or one column is filled, a new row or column is started.
- You can specify the way the nodes are placed horizontally or vertically using one of two constants: **Orientation.HORIZONTAL** or **Orientation.VERTICAL**.
- You can also specify the gap between the nodes in pixels.

FLOW PANE



Horizontal Orientation



Vertical Orientation

FLOW PANE

javafx.scene.layout.FlowPane

-alignment: `ObjectProperty<Pos>`
-orientation:
 `ObjectProperty<Orientation>`
-hgap: `DoubleProperty`
-vgap: `DoubleProperty`

+`FlowPane()`
+`FlowPane(hgap: double, vgap: double)`
+`FlowPane(orientation: ObjectProperty<Orientation>)`
+`FlowPane(orientation: ObjectProperty<Orientation>, hgap: double, vgap: double)`

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The overall alignment of the content in this pane (default: `Pos.LEFT`).
The orientation in this pane (default: `Orientation.HORIZONTAL`).

The horizontal gap between the nodes (default: 0).

The vertical gap between the nodes (default: 0).

Creates a default `FlowPane`.

Creates a `FlowPane` with a specified horizontal and vertical gap.

Creates a `FlowPane` with a specified orientation.

Creates a `FlowPane` with a specified orientation, horizontal gap and vertical gap.

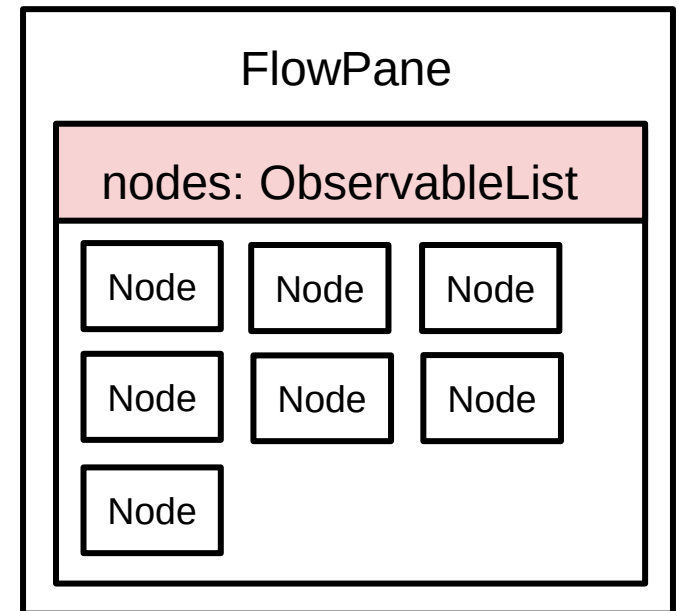
ShowFlowPane

FLOW PANE

- Data fields alignment, orientation, hgap, and vgap are binding properties.
- For a data field of `ObjectProperty<T>` type, the value getter method returns a value of type **T**, and the property getter method returns a property value of type `ObjectProperty<T>`.

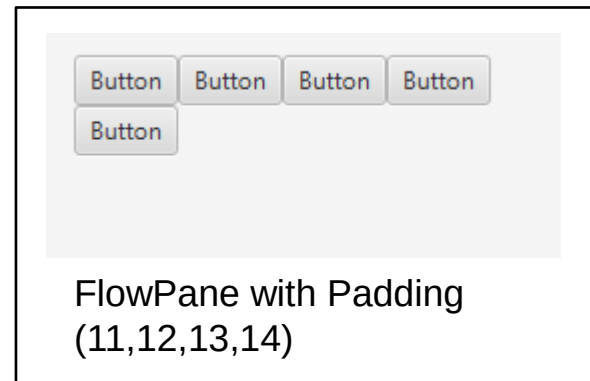
FLOW PANE

- Each **FlowPane** contains an object of **ObservableList** for holding the nodes.
- This list can be obtained using the **getChildren()** method.
- To add a node into a FlowPane is to add it to this list using the following methods:
 - `add(node)`
 - `addAll(node1, node2, ...)`
- **Important Note:**
 - Adding a node to a pane multiple times or to different panes will cause a runtime error.



FLOW PANE

- You can set the FlowPane padding property with an Insets object.
- An Insets object specifies the size of the border of a pane.
- The constructor `Insets(11, 12, 13, 14)` creates an Insets with the border sizes for top (11), right (12), bottom (13), and left (14) in pixels.



FLOW PANE

ShowFlowPane

// Create a pane and set its properties

```
FlowPane pane = new FlowPane();  
pane.setPadding(new Insets(11, 12, 13, 14));  
pane.setHgap(5);  
pane.setVgap(5);
```

// Place nodes in the pane

```
pane.getChildren().addAll(new Label("First Name:"),  
    new TextField(), new Label("MI:"));  
TextField tfMi = new TextField();  
tfMi.setPrefColumnCount(1);  
pane.getChildren().addAll(tfMi, new Label("Last Name:"),  
    new TextField());
```

FLOW PANE EXAMPLE



A screenshot of a Java Swing window titled "ShowFlowPane". The window contains a FlowPane widget with three text input fields arranged horizontally. The first field is labeled "First Name:", the second is labeled "MI:", and the third is labeled "Last Name:". The "MI:" label is positioned between the first and second input fields. The window has standard Mac OS X window controls (red, yellow, and green buttons) in the top-left corner and standard window controls (minimize, maximize, and close buttons) in the top-right corner.

GRID PANE

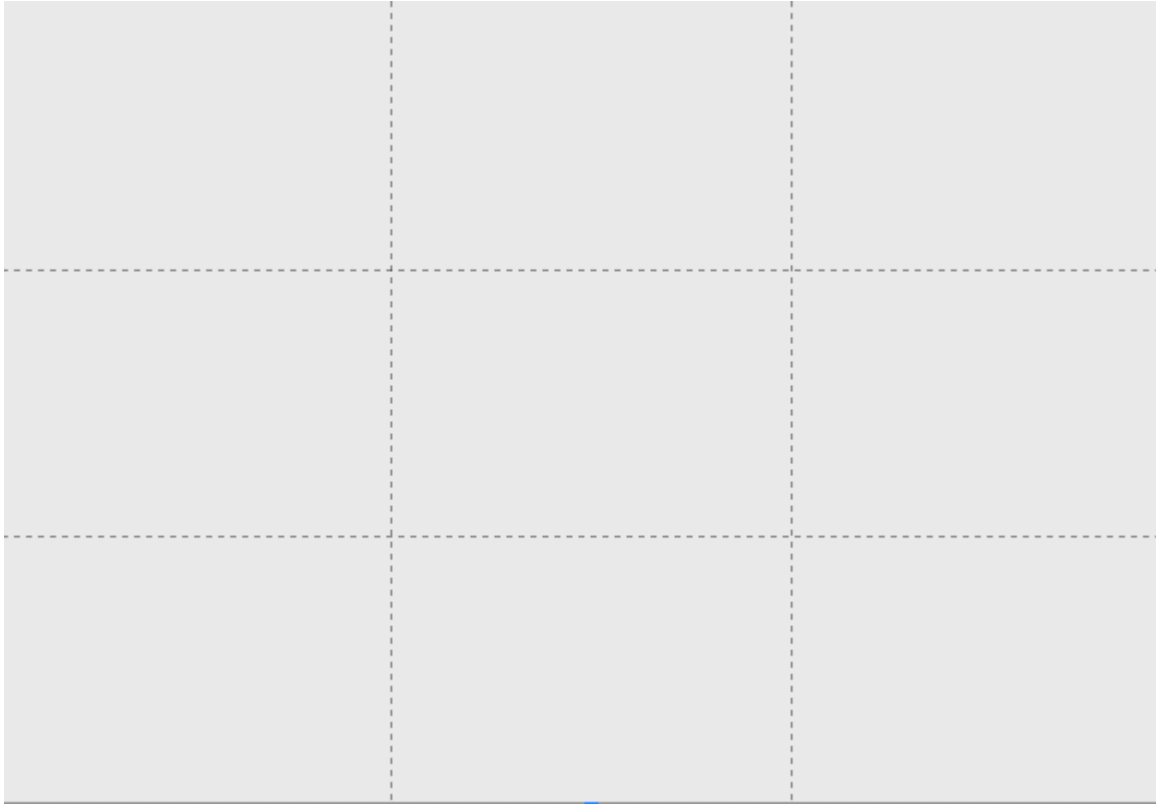
- A GridPane arranges nodes in a grid (matrix) formation.
- The nodes are placed in the specified column and row indices.
- To remove a node from a GridPane, use:

```
pane.getChildren().remove(node)
```

- To remove all nodes, use:

```
pane.getChildren().removeAll()
```

GRID PANE



GRID PANE

javafx.scene.layout.GridPane

-alignment: ObjectProperty<Pos>
-gridLinesVisible: BooleanProperty
-hgap: DoubleProperty
-vgap: DoubleProperty

+GridPane()
+add(child: Node, columnIndex: int, rowIndex: int): void
+addColumn(columnIndex: int, children: Node...): void
+addRow(rowIndex: int, children: Node...): void
+getColumnIndex(child: Node): int
+setColumnIndex(child: Node, columnIndex: int): void
+getRowIndex(child: Node): int
+setRowIndex(child: Node, rowIndex: int): void
+setHalignment(child: Node, value: HPos): void
+setValignment(child: Node, value: VPos): void

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The overall alignment of the content in this pane (default: Pos.LEFT).

Is the grid line visible? (default: false)

The horizontal gap between the nodes (default: 0).

The vertical gap between the nodes (default: 0).

Creates a GridPane.

Adds a node to the specified column and row.

Adds multiple nodes to the specified column.

Adds multiple nodes to the specified row.

Returns the column index for the specified node.

Sets a node to a new column. This method repositions the node.

Returns the row index for the specified node.

Sets a node to a new row. This method repositions the node.

Sets the horizontal alignment for the child in the cell.

Sets the vertical alignment for the child in the cell.

GRID PANE

- By default, the grid pane will resize rows and columns to the preferred sizes of its contents, even if the grid pane is resized larger than its preferred size.
- You may purposely set a large value for the preferred width and height of its contents by invoking the `setPrefWidth` and `setPrefHeight` methods, so the contents will be automatically stretched to fill in the grid pane when the grid pane is enlarged.

ShowGridPane

GRID PANE

ShowGridPane

// Create a pane and set its properties

```
GridPane pane = new GridPane();  
pane.setAlignment(Pos.CENTER);  
pane.setPadding(new Insets(11.5, 12.5, 13.5, 14.5));  
pane.setHgap(5.5);  
pane.setVgap(5.5);
```

// Place nodes in the pane

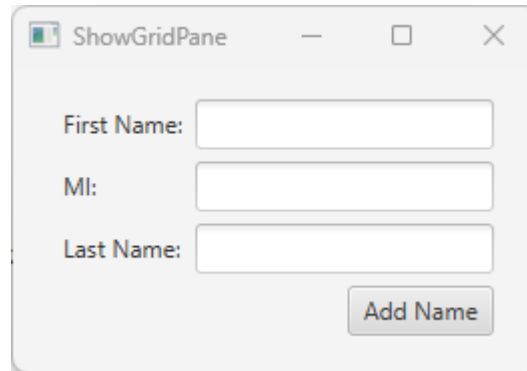
```
pane.add(new Label("First Name:"), 0, 0);  
pane.add(new TextField(), 1, 0);  
pane.add(new Label("MI:"), 0, 1);  
pane.add(new TextField(), 1, 1);  
pane.add(new Label("Last Name:"), 0, 2);
```

1

```
pane.add(new TextField(), 1, 2);  
Button btAdd = new Button("Add Name");  
pane.add(btAdd, 1, 3);  
GridPane.setHalignment(btAdd, HPos.RIGHT);
```

2

GRID PANE EXAMPLE



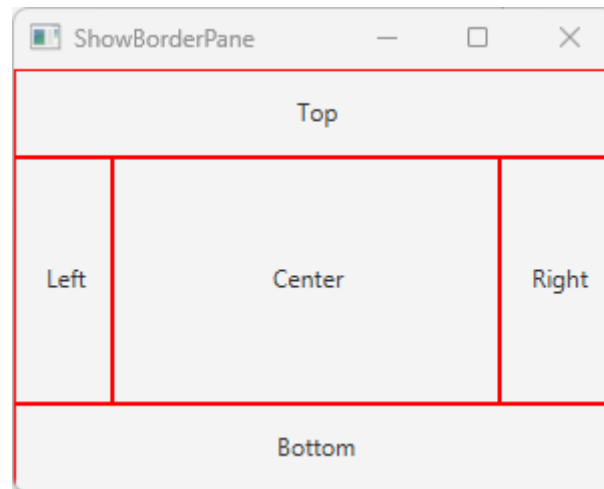
First Name:

MI:

Last Name:

BORDER PANE

- Places the nodes in the top, right, bottom, left, and center regions.



BORDER PANE

javafx.scene.layout.BorderPane

-top: ObjectProperty<Node>
-right: ObjectProperty<Node>
-bottom: ObjectProperty<Node>
-left: ObjectProperty<Node>
-center: ObjectProperty<Node>

+BorderPane()
+setAlignment(child: Node, pos: Pos)

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The node placed in the top region (default: null).
The node placed in the right region (default: null).
The node placed in the bottom region (default: null).
The node placed in the left region (default: null).
The node placed in the center region (default: null).

Creates a BorderPane.

Sets the alignment of the node in the BorderPane.

BORDER PANE

- **Important Notes:**
 - To remove a node from the top region, invoke `setTop(null)`.
 - If a region is not occupied, no space will be allocated for this region.

ShowBorderPane

BORDER PANE

ShowBorderPane

// Create a border pane

```
BorderPane pane = new BorderPane();
```

// Place nodes in the pane

```
pane.setTop(new CustomPane("Top"));
```

```
pane.setRight(new CustomPane("Right"));
```

```
pane.setBottom(new CustomPane("Bottom"));
```

```
pane.setLeft(new CustomPane("Left"));
```

```
pane.setCenter(new CustomPane("Center"));
```

BORDER PANE

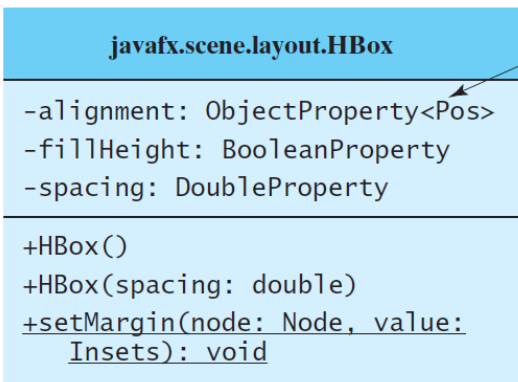
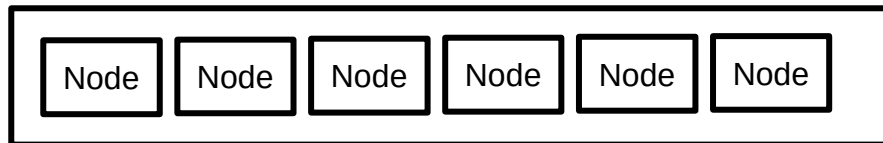
CustomPane

// Define a custom pane to hold a label in the center of the pane

```
class CustomPane extends StackPane {  
    public CustomPane(String title) {  
        getChildren().add(new Label(title));  
        setStyle("-fx-border-color: red");  
        setPadding(new Insets(11.5, 12.5, 13.5, 14.5));  
    }  
}
```

HBOX PANE

- Places the nodes in a single row.



The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

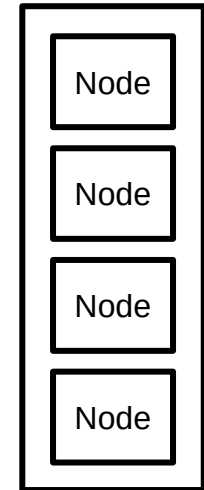
The overall alignment of the children in the box (default: Pos.TOP_LEFT).
Is resizable children fill the full height of the box (default: true).
The horizontal gap between two nodes (default: 0).

Creates a default HBox.

Creates an HBox with the specified horizontal gap between nodes.
Sets the margin for the node in the pane.

VBOX PANE

- Places the nodes in a single column.



<code>javafx.scene.layout.VBox</code>
<code>-alignment: ObjectProperty<Pos></code> <code>-fillWidth: BooleanProperty</code> <code>-spacing: DoubleProperty</code>
<code>+VBox()</code> <code>+VBox(spacing: double)</code> <code>+setMargin(node: Node, value: Insets): void</code>

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The overall alignment of the children in the box (default: `Pos.TOP_LEFT`).
Is resizable children fill the full width of the box (default: `true`).
The vertical gap between two nodes (default: `0`).

Creates a default `VBox`.
Creates a `VBox` with the specified horizontal gap between nodes.
Sets the margin for the node in the pane.

ShowHBoxVBox

VBOX PANE

ShowHBoxVBox

// Create a border pane

```
BorderPane pane = new BorderPane();
```

// Place nodes in the pane

```
pane.setTop(getHBox());
```

```
pane.setLeft(getVBox());
```

VBOX PANE

getHBox

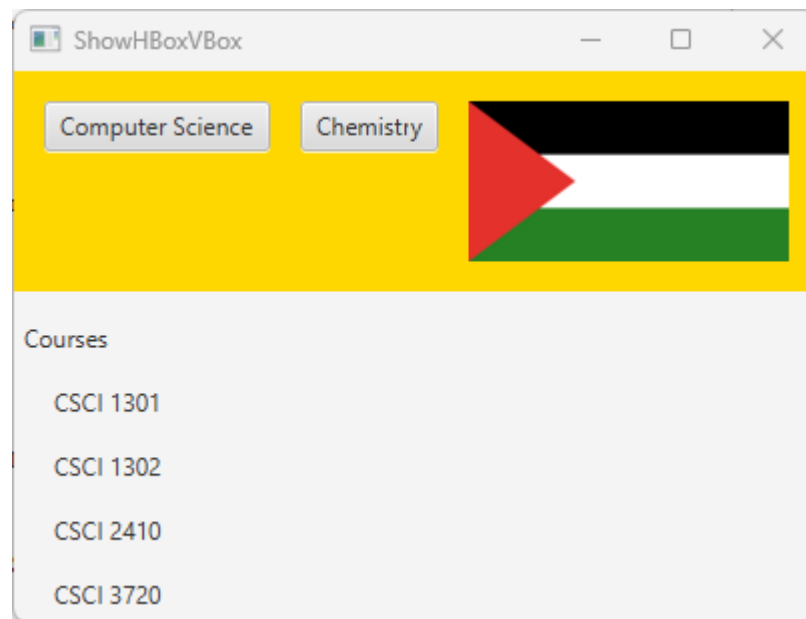
```
private HBox getHBox() {  
    HBox hBox = new HBox(15);  
    hBox.setPadding(new Insets(15, 15, 15, 15));  
    hBox.setStyle("-fx-background-color: gold");  
    hBox.getChildren().add(new Button("Computer Science"));  
    hBox.getChildren().add(new Button("Chemistry"));  
    ImageView imageView = new ImageView(new Image("image/us.gif"));  
    hBox.getChildren().add(imageView);  
    return hBox;  
}
```

VBOX PANE

getVBox

```
private VBox getVBox() {  
    VBox vbox = new VBox(15);  
    vbox.setPadding(new Insets(15, 5, 5, 5));  
    vbox.getChildren().add(new Label("Courses"));  
  
    Label[] courses = {new Label("CSCI 1301"), new Label("CSCI 1302"),  
        new Label("CSCI 2410"), new Label("CSCI 3720")};  
  
    for (Label course: courses) {  
        vbox.setMargin(course, new Insets(0, 0, 0, 15));  
        vbox.getChildren().add(course);  
    }  
  
    return vbox;  
}
```

HBOX & VBox PANE EXAMPLE



**What is the Difference Between
The HBOX/VBOX and the FlowPane?**

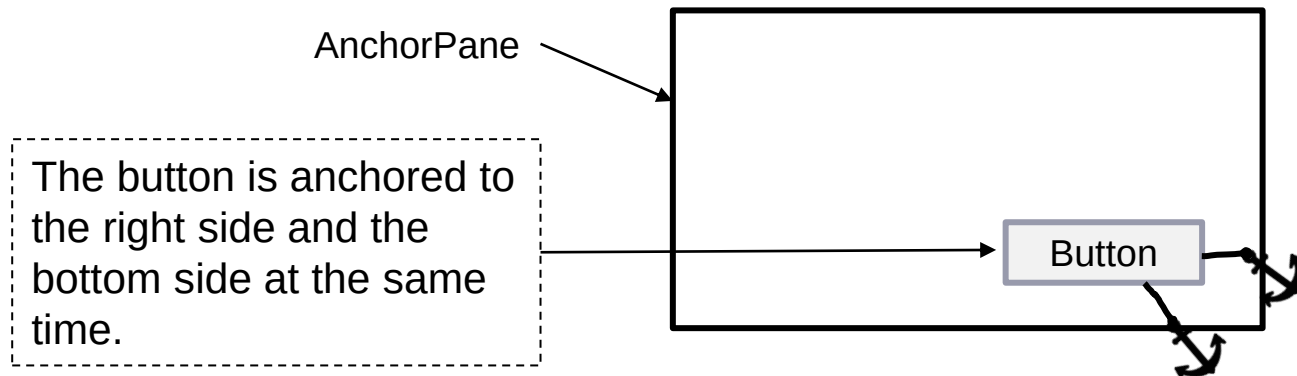
THE DIFFERENCE

- An HBox lays out its children in a **single** horizontal row.
- A VBox lays out its children in a **single** vertical column.
- Recall that a FlowPane can lay out its children in **multiple** rows or multiple columns, but an HBox or a VBox can lay out children only in one row or one column.

ANCHOR PANE



- The `AnchorPane` layout pane enables you to anchor nodes to the top, bottom, left side, right side, or center of the pane.
- As the window is resized, the nodes maintain their position relative to their anchor point.
- Nodes can be anchored to more than one position and more than one node can be anchored to the same position.



ANY QUESTION?